구조: 관계형 DB 읽기와 한 테이블 질의

메타데이터론 3강 ① · 데이터베이스와 SQL

2강 'XML과 JSON'에 이어 · 다음 주 ②: JOIN·집계·데이터 바꾸기

```
FROM 메타데이터론
```

```
WHERE 차시 = '3강 ①'
```

```
SELECT 구조, 질의
```

2026년 2학기 · 전남대학교 문헌정보학과 부교수 박주현 · park51566@jnu.ac.kr

2강에서 이어서: 그릇에서 창고로

2 2강: 그릇 (XML · JSON)

```
{ "title": "어린 왕자",
  "creators": [ … ],
  "publisher": "문학동네" }
```

레코드 한 건을 통째로 담아 주고받는다

3 3강: 창고 (데이터베이스)

book | **author**

item | **loan**

수만 건을 표로 나눠 담고 · 잇고 · 묻고 · 지킨다

오늘: 표로 나눠 담은 구조를 읽고, SQL로 표 하나에 묻는다

2강 덱 4부(JSON 문법)의 『어린 왕자』 레코드를 줄여 보임 · '…'는 생략

두 주 내내 쓸 DB: 작은 도서관

book 31행 도서 (판 하나)	**author** 47행 저자·역자	**book_author** 53행 도서↔사람 잇기
publisher 20행 출판사	**kdc** 17행 KDC 분류	**item** 55행 소장 자료 (실물)
member 20행 이용자 (가상)	**loan** 116행 대출 기록 (가상)	**reservation** 11행 예약 (가상)

『어린 왕자』 세 종(문학동네 2007 · 열린책들 2015 · Harcourt 2000)도 들어 있다

library.sql (테이블 9개) · 이용자·대출·예약은 가상, 서지는 실제 책을 참고해 단순화 · 기준일 2026-11-05

목차

1 **왜 데이터베이스인가**
파일·스프레드시트·XML/JSON과 비교 · DBMS

2 **데이터 모델**
계층 · 관계 · 문서 · 그래프

3 **관계형 모델의 말**
테이블 · 행 · 열 · 도메인 · NULL

4 **키**
기본 키 · 후보 키 · 외래 키

5 **ERD와 관계**
1:N · N:M · 잇는 표 → 실습 1 · 쉬는 시간

6 **정규화**
갱신 이상 · 1NF~3NF

7 **SQL 개요**
DDL · DML

8 **SELECT**
WHERE · ORDER BY · LIKE · NULL → 실습 2 · 생각 1

다음 주 · 3강 ② 잇기
JOIN · GROUP BY · 하위 질의 · 데이터 바꾸기 ·
메타데이터 관점 · 실습 3~5

1~5부: 구조 읽기 → 실습 1 → 쉬는 시간 → 6~8부: 정규화와 SQL → 실습 2 · 생각 1

같은 자료, 네 가지 보관법

보관법	예	잘하는 것	힘든 것
문서 파일	한글 파일 대출 장부	사람이 읽기	세기 · 찾기 · 고치기
스프레드시트	엑셀 한 시트	정렬 · 필터 · 작은 표	같은 값 되풀이 · 동시 수정 · 시트 잇기
XML · JSON 파일	2강 book.xml · book.json	레코드 한 건을 구조째 교환	많은 건에서 조건 검색 · 함께 고치기
데이터베이스	오늘의 작은 도서관	**나눠 담고 · 잇고 · 묻고 · 지키기**	처음에 설계가 필요

DB는 '많은 레코드를 여럿이 함께, 오래, 틀리지 않게' 다루려고 쓴다

한 시트에 다 적으면 생기는 일

대출 시트 (한 장에 펼친 모습)

대출일	이용자	소속	서명	출판사
2026-09-06	서유진	사회학과	소년이 온다	창비
2026-09-08	오수아	기록관리학협동과정	소년이 온다	창비
2026-09-12	이도윤	국어국문학과	소년이 온다	창비
2026-09-22	임서연	문헌정보학과	소년이 온다	창비

같은 값이 줄마다
서명 · 출판사 · 이용자 정보 되풀이

고칠 때 어긋남
한 줄만 빠뜨려도 두 가지 사실

동시에 고치면
누구 것이 남나?

해법: 사실 하나는 한 곳에만 적고, 번호로 가리킨다 → 6부 정규화

작은 도서관 DB의 대출 2·4·15·41번을 loan·member·item·book·publisher에서 모아 한 장으로 펼침 (이용자는 가상)

DBMS: 데이터를 맡아 지키는 프로그램



DBMS 제품의 예: SQLite · MySQL · PostgreSQL · Oracle · SQL Server

오늘 쓰는 DBMS: 브라우저 안의 SQLite

1

홈페이지 실습실을 연다

`lab.html`

→

2

브라우저 안에 DB가 만들어진다

`sql.js` = 브라우저용 SQLite

→

3

SQL을 쓰고 '실행'

결과 표가 바로 나온다

→

4

망가져도 괜찮다

'DB 초기화' → 처음 상태

설치 없음 · 가입 없음 · 내 브라우저에만

바꾼 자료와 맞힌 문제 기록은 내 브라우저에만 남고 다른 사람에게 보이지 않는다. 크롬 · 엣지 · 사파리 최신판이면 된다.

출처: 실습 자료 페이지 parkjuhyun-jnu.github.io/materials/metadata-2026-2/sql-db/ 안내문

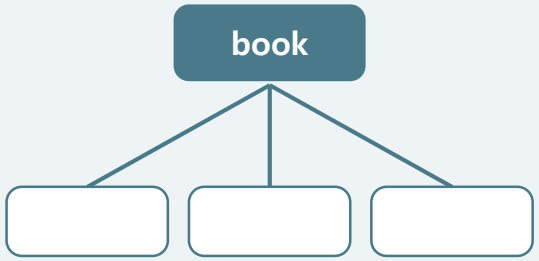
데이터 모델: 세상을 담는 틀

개념	**무엇이 있고 어떻게 이어지나**	도서 한 종에 실물이 여러 권	**ERD (5부)**
논리	**어떤 표 · 열 · 키로 담나**	`book · item(book_id → book)`	**테이블 (3·4부)**
물리	**어떻게 저장하나**	파일 · 색인	**DBMS가 맡음**

메타데이터 설계 = 개념(무엇을 기술하나) + 논리(어떤 칸에 담나)

네 가지 데이터 모델과 2강

계층



나무: 부모 아래 자식

2강 XML 문서

관계

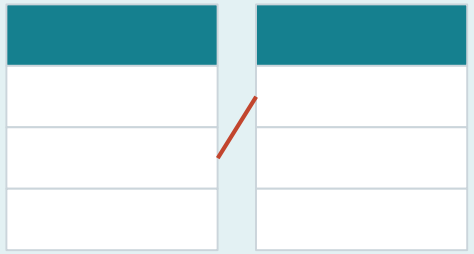
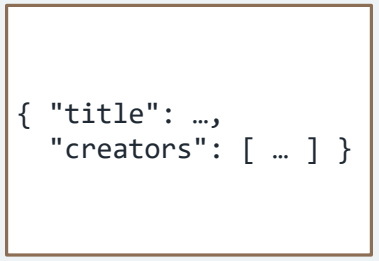


표: 행과 열, 같은 값으로 잇기

오늘 · SQL

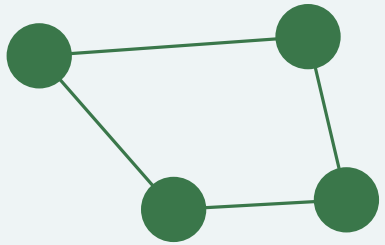
문서



JSON 덩어리째 저장

2강 JSON · 문서형 DB

그래프



노드와 화살표

2강 RDF · SPARQL

오늘의 주인공: 관계형 모델 — 모든 것을 표로, 표끼리는 같은 값으로

같은『어린 왕자』, 덩어리와 표

문서 모양: 한 덩어리 (수업용으로 다시 적음)

```
{
  "book_id": 9,
  "title": "어린 왕자",
  "publisher": "문학동네",
  "pub_year": 2007,
  "creators": [
    { "name": "앙투안 드 생텍쥐페리",
      "role": "지은이" },
    { "name": "김화영",
      "role": "옮긴이" }
  ]
}
```

book

book_id	title	publisher_id	pub_year
9	어린 왕자	2	2007

book_author

book_id	author_id	role
9	7	지은이
9	8	옮긴이

publisher

publisher_id	name
2	문학동네

author

author_id	name
7	앙투안 드 생텍쥐페리
8	김화영

표 모양: 한 번씩 적고 번호로 잇는다

값: library.sql의 book 9번 · book_author · author 7·8번 · publisher 2번 (일부 열만 보임)

테이블 · 행 · 열

↓ 열(column) = 속성 · 같은 종류의 값

book 테이블 (31행 가운데 8~12번)

book_id	title	publisher_id	pub_year	kdc_code	pages	language
8	난장이가 쏘아올린 작은 공	7	2000	813	*NULL*	kor
9	어린 왕자	2	2007	863	160	kor
10	어린 왕자	8	2015	863	152	kor
11	데미안	3	2000	853	230	kor
12	1984	3	2003	843	460	kor

한 칸에 값 하나 · 행에는 순서가 없다

← 행(row) = 레코드 하나 (도서 한 종)

library.sql book 테이블의 8~12번 행 (빨간 테두리: 9번 행)

같은 것을 부르는 네 가지 이름

관계형 모델 (이론)	SQL · DBMS	스프레드시트	메타데이터
릴레이션 (relation)	테이블 (table)	시트	레코드 묶음 (목록 파일)
튜플 (tuple)	행 (row)	줄	레코드 하나
속성 (attribute)	열 (column)	머리 칸	요소 · 필드
도메인 (domain)	자료형 · 제약	셀 서식	값 규칙 · 통제 어휘

책마다 이름이 섞여 나온다 — 같은 것을 가리킨다

스키마: 테이블의 설계도

library.sql — book 테이블 (주석 생략)

```
CREATE TABLE book (
  book_id      INTEGER PRIMARY KEY,
  title        TEXT NOT NULL,
  publisher_id INTEGER REFERENCES publisher(publisher_id),
  pub_year     INTEGER,
  kdc_code     TEXT REFERENCES kdc(code),
  pages        INTEGER,
  language     TEXT NOT NULL DEFAULT 'kor'
);
```

PRIMARY KEY
기본 키 (4부)

REFERENCES
외래 키: 다른 표를 가리킨다

NOT NULL
비워 둘 수 없다

DEFAULT 'kor'
안 적으면 kor

스키마 = 틀(요소 정의서) · 행 = 내용(레코드)

도메인: 칸에 들어갈 수 있는 값

열	자료형	값의 범위 (도메인)	메타데이터로 치면
book.pub_year	INTEGER	정수 (연도)	날짜 형식 규칙
book.language	TEXT	ISO 639-2 세 글자: kor · eng	언어 코드표
book.kdc_code	TEXT	kdc 테이블에 있는 기호만	분류표
item.status	TEXT	대출가능 · 대출중 · 분실	통제 어휘

같은 뜻은 같은 값으로 — 그래야 정확히 찾고 센다

library.sql의 열 정의와 주석 · status 값은 약속(CHECK 제약은 없음)

NULL: 값이 비어 있다는 표시

모른다

author 13번 정회성

name	birth_year
정회성	*NULL*

아직 없다

loan 95번

loan_id	return_date
95	*NULL*

해당 없다

author 8번 김화영

name	name_original
김화영	*NULL*

NULL ≠ 0 ≠ '' · 찾을 때는 = NULL 이 아니라 IS NULL (8부)

값: library.sql author 8·13번, loan 95번 · 비어 있는 까닭의 구분은 수업용 설명(DB에는 모두 NULL)

기본 키(PK): 행을 하나로 가리키는 값

item: 『어린 왕자』 문학동네판(book 9번)의 실물 두 권

item_id	book_id	location	status
EM000234	9	2층 인문자료실	대출중
EM000241	9	2층 인문자료실	대출가능

← 서명·위치가 같아도 등록번호로 가린다

기본 키의 조건

- 겹치지 않는다 (유일)
- 비어 있지 않다 (NOT NULL)
- 되도록 바뀌지 않는다

도서관의 등록번호 = 개별자료(item)의 기본 키

값: library.sql item 테이블 (EM000234 · EM000241)

후보 키 · 기본 키 · 대체 키

library.sql — publisher (주석 생략)

```
CREATE TABLE publisher (
  publisher_id INTEGER PRIMARY KEY,
  name         TEXT NOT NULL UNIQUE,
  city         TEXT
);
```

publisher 앞 3행

publisher_id	name	city
1	창비	파주
2	문학동네	파주
3	민음사	서울

후보 키
publisher_id · name
둘 다 겹치지 않는다

기본 키
publisher_id
그중 고른 하나

대체 키
name (UNIQUE)
고르지 않은 후보

대리 키 vs 자연 키

일련번호(publisher_id) vs ISBN·등록번호처럼 세상에 있던 값

복합 키: 열 여럿을 묶어 하나의 키로

library.sql — book_author (주석 생략)

```
CREATE TABLE book_author (
  book_id   INTEGER REFERENCES book(book_id),
  author_id INTEGER REFERENCES author(author_id),
  role      TEXT NOT NULL,
  seq       INTEGER NOT NULL DEFAULT 1,
  PRIMARY KEY (book_id, author_id, role)
);
```

book_author 가운데 4행

book_id	author_id	role	seq
9	7	지은이	1
9	8	옮긴이	1
10	7	지은이	1
31	7	지은이	1

값: library.sql book_author (book 9·10·31번)

왜 묶어야 하나

- book_id만: 9번에 두 사람 → 겹침
- author_id만: 7번이 세 책 → 겹침
- **(book_id, author_id, role) 셋을 묶으면 한 행**
- role까지 넣은 까닭: 한 사람이 같은 책에 두 역할

외래 키(FK): 다른 표의 기본 키를 가리킨다

item (FK: book_id)

item_id	book_id	location
EM000234	9	2층 인문자료실
EM000241	9	2층 인문자료실
EM000248	10	2층 인문자료실

book (PK: book_id)

book_id	title	pub_year
9	어린 왕자	2007
10	어린 왕자	2015

없는 번호는 못 넣는다
book에 없는 99번을 item에 적으면 거부

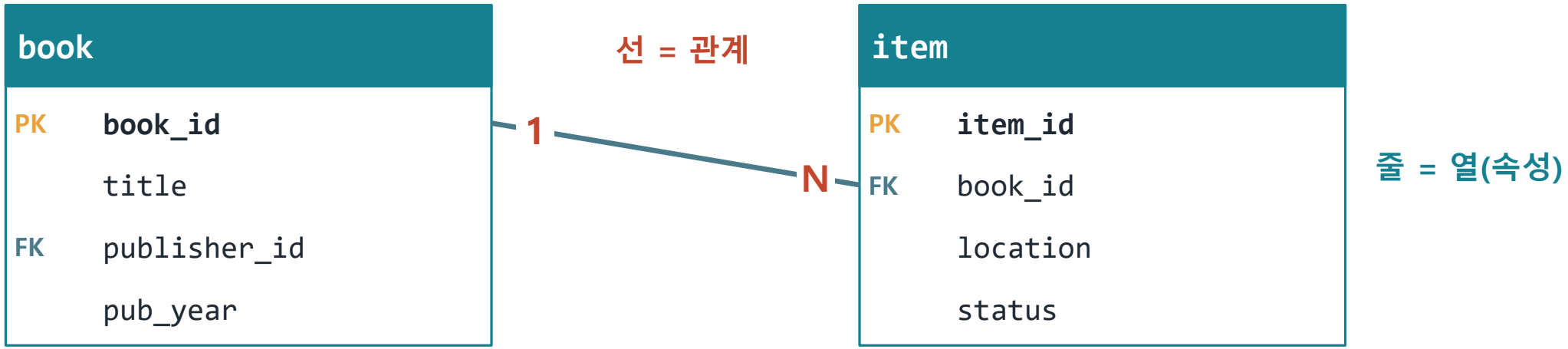
가리키는 행은 못 지운다
item이 9번을 가리키는 동안 book 9번 삭제 거부

2강: 저자 칸에 이름 대신 식별자(URI) ↔ DB: 칸에 값 대신 다른 표의 키

값: library.sql item·book · 2강 ① 5번(국립중앙도서관 LOD KMO200722491의 저자 → KAC199623935)

ERD 읽는 법: 상자 · 열 · 선

상자 = 테이블(개체)



PK 기본 키 FK 외래 키

1 —— N : book 한 행에 item 여러 행 (도서 한 종, 실물 여러 권)

표기는 실습 자료 '구조 보기'(structure.html)와 같게 1·N으로 적음

관계의 세 가지: 1:1 · 1:N · N:M

1 : 1

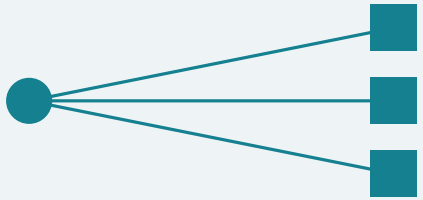
한 행 ↔ 한 행



실습 DB에는 없음
(예: 사람 ↔ 여권 번호)

1 : N

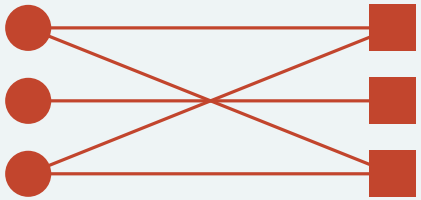
한 행 ↔ 여러 행



`publisher → book`
`book → item · item → loan`
`member → loan`

N : M

여러 행 ↔ 여러 행



`book ⇔ author`
(잇는 표 `book_author`)

관계형 DB는 N:M을 바로 적지 못한다 → 잇는 표를 둔다

1:N 따라가기: 도서 한 종 → 실물 → 대출

book (1행)

book_id	title
9	어린 왕자

item (book_id = 9 → 2행)

item_id	book_id	status
EM000234	9	대출중
EM000241	9	대출가능

loan (item_id가 둘 중 하나 → 5행)

loan_id	item_id	loan_date	return_date
5	EM000234	2026-09-08	2026-09-20
25	EM000241	2026-09-15	2026-09-24
61	EM000234	2026-10-02	2026-10-16
77	EM000241	2026-10-11	2026-10-27
100	EM000234	2026-10-23	*NULL*

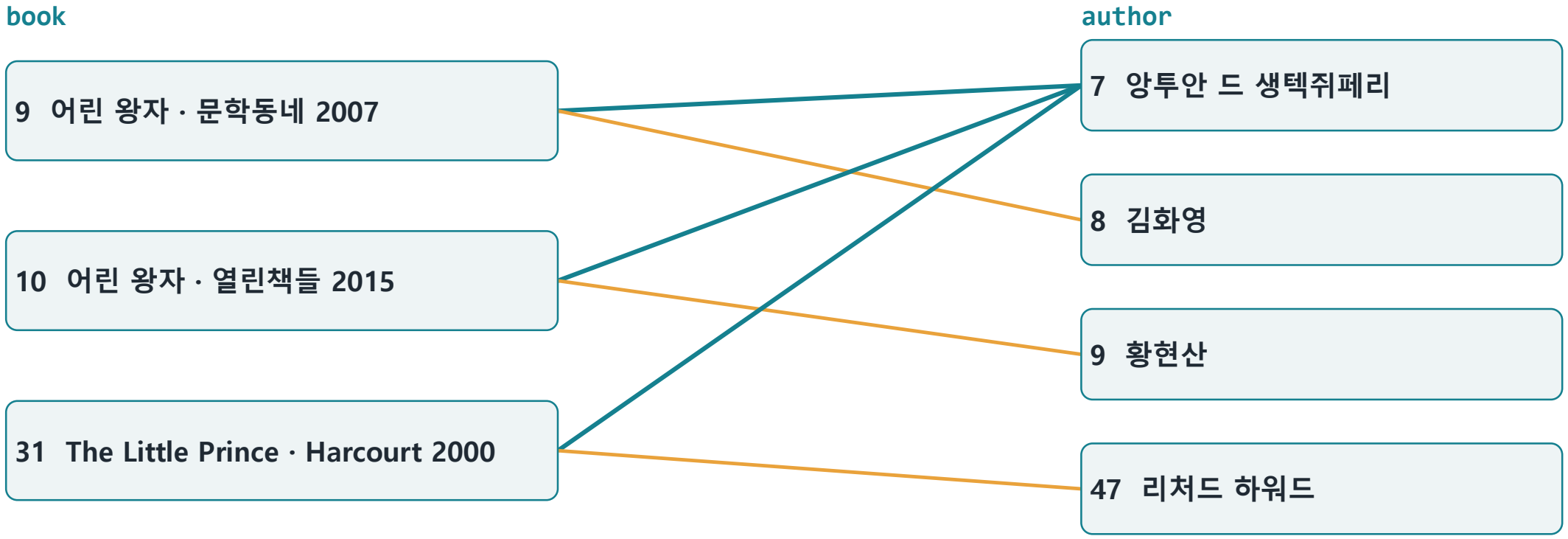
1 → 2 → 5

도서 한 종
실물 두 권
대출 다섯 건

100번:
반납일 NULL
→ 아직 대출 중

값: library.sql book 9 · item · loan (대출일 순으로 놓음)

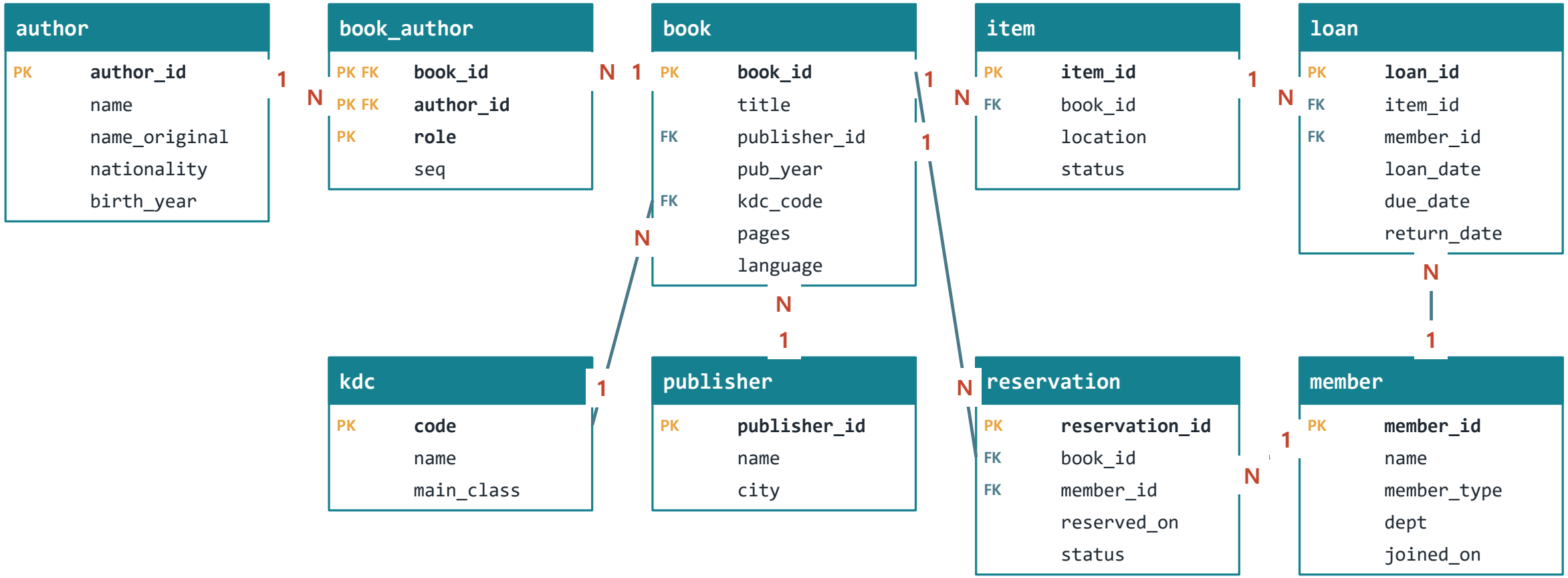
N:M은 잇는 표로: book_author



역할(role)을 잇는 표에 — MARC 100·700, DC creator·contributor와 같은 생각

값: library.sql book_author의 book 9·10·31번 6행 · author 7·8·9·47번

작은 도서관 전체 ERD



대출(loan)은 실물(item)에 · 예약(reservation)은 도서(book)에 붙는다

library.sql 테이블 9개 · PK 기본 키, FK 외래 키 · 선의 1 쪽 한 행에 N 쪽 여러 행

실습 55분 · 브라우저 + 인터넷

실습 안내: 준비물 · 순서 · 시간

순서	내용	분	페이지
실습 1	DB 구조 읽기: PK·FK 찾기 · 『소년이 온다』 따라가기	15	구조 보기
실습 2	SELECT 기초 8문제 (1-1 ~ 1-8) · 쉬는 시간 뒤	35	실습실
생각 1	왜 표를 나눌까 (짝 토론 · 채점 없음)	5	—

실습 1 = 지금 · 실습 2와 생각 1 = 쉬는 시간과 6~8부 뒤

준비물

노트북 + 인터넷

크롬 · 엣지 · 사파리 최신판

설치 · 가입 없음

기록은 내 브라우저에만

실습 자료 parkjuhyun-jnu.github.io/materials/metadata-2026-2/sql-db/

실습실 lab.html · 구조 보기 structure.html · 브라우저 안에서 SQLite(sql.js)가 돈다 — 설치·가입 없음

실습 1 · 15분

실습 1: DB 구조 읽기 — 『소년이 온다』 따라가기

1. '구조 보기' 열기: 상자를 누르면 열 설명과 앞 10행
2. 아홉 테이블의 기본 키(PK)를 적는다
3. 선마다 1 쪽과 N 쪽을 확인한다
4. book에서 『소년이 온다』의 book_id 찾기
5. item에서 그 book_id의 실물 수 · 위치 · 상태
6. loan에서 등록번호마다 대출 건수 (아래 한 줄)

채울 것	답
『소년이 온다』 book_id	
실물(item) 수 · 위치	
대출(loan) 건수	
아직 반납 안 된 대출	

맛보기: 실습실에 붙여 넣고 '등록번호'만 바꾸기

```
SELECT * FROM loan WHERE item_id = '등록번호';
```

실습 자료 parkjuhyun-jnu.github.io/materials/metadata-2026-2/sql-db/

실습실 lab.html · 구조 보기 structure.html · 브라우저 안에서 SQLite(sql.js)가 돈다 — 설치·가입 없음

쉬는 시간 10분

다음: 6부 정규화 → 7·8부 SQL과 SELECT → 실습 2 · 생각 1

쉬는 동안 해 볼 것: SQL 실습실 열어 두기 (lab.html)
parkjuhyun-jnu.github.io/materials/metadata-2026-2/sql-db/

한 표에 다 적으면: 같은 사실이 네 번

한 표에 다 적은 경우 (가정)

title	출판사	소재지
채식주의자	창비	파주
소년이 온다	창비	파주
작별하지 않는다	문학동네	파주
아몬드	창비	파주
나의 문화유산답사기 1	창비	파주

정규화 = 같은 사실은 한 곳에만 적도록 표를 나누는 일

book (번호만)

title	publisher_id
채식주의자	1
소년이 온다	1
작별하지 않는다	2
아몬드	1
나의 문화유산답사기 1	1

publisher (한 번만)

publisher_id	name	city
1	창비	파주
2	문학동네	파주

값: library.sql book 1·2·3·5·24번과 publisher 1·2번 · 왼쪽 표는 설명용으로 합쳐 보인 것

나누지 않으면 생기는 세 가지 이상

갱신 이상

창비 소재지를 바꾸려면 네 줄을 모두 고친다

↓

한 줄만 빠뜨려도 '창비는 두 곳'

삽입 이상

아직 책이 없는 출판사

↓

적을 자리가 없다 (책 칸을 비워 둘 수도 없다)

삭제 이상

이성과힘의 책은
『난장이가 쏘아올린 작은 공』 하나

↓

그 책을 지우면 출판사 정보도 사라진다

까닭: 서로 다른 사실(도서 · 출판사)을 한 행에 섞어 적었다

이성과힘(publisher 7번)의 도서는 library.sql에서 book 8번 하나

제1정규형(1NF): 한 칸에 값 하나

× 한 칸에 여럿 (가정)

book_id	title	사람
9	어린 왕자	생텍쥐페리(지은이), 김화영(옮긴이)

× 칸을 늘리기 (가정) — 셋째 사람은?

book_id	title	사람1	사람2
9	어린 왕자	생텍쥐페리	김화영



○ book_author: 한 행에 한 사람

book_id	author_id	role
9	7	지은이
9	8	옮긴이

사람이 몇 명이든 행만 늘어난다

1NF: 한 칸에 값 하나 · 같은 종류의 칸을 늘리지 않는다

값: library.sql book 9번 · book_author · author 7·8번 · × 두 표는 설명용 가정

제2·3정규형: 키 전체에, 키에만 기대기

2NF: 키 전체에 기대기

book_id	author_id	role	국적?
9	7	지은이	프랑스

국적은 author_id만으로 정해진다 (키의 일부)
→ author 표로

3NF: 키에만 기대기

book_id	publisher_id	소재지?
2	1	파주

소재지는 publisher_id를 거쳐 정해진다 (이행)
→ publisher 표로

키에, 키 전체에, 키에만 기댄다 — 실습 DB는 3NF까지

빨간 열은 설명용으로 덧붙인 가정 · 실제 값: author 7번 nationality, publisher 1번 city

정규화와 전거 통제는 같은 생각

DB 정규화	**author 표에 사람을 한 번**	다른 표는 author_id로 가리킨다
전거 통제	**전거 레코드에 이름을 한 번**	서지 레코드는 전거를 가리킨다
링크드 데이터 (2강)	**사람마다 URI 하나**	레코드는 URI로 가리킨다

한 번만 적고, 나머지는 가리킨다

반정규화: 읽기를 빠르게 하려고 일부러 합쳐 적기도

SQL: 무엇을 원하는지 말하는 언어

보여 달라 `SELECT name, birth_year`

어디에서 `FROM author`

어떤 것만 `WHERE nationality = '미국'`

SQL

```sql
SELECT name, birth_year
FROM   author
WHERE  nationality = '미국';
```

어떻게 찾을지는 DBMS가 정한다

파일을 어떤 순서로 읽을지, 색인을 쓸지 = DBMS의 일

선언형: '무엇을'만 말한다 · 표준이 있지만 DBMS마다 사투리가 있다

SQL 문장의 네 갈래

갈래	하는 일	대표 문장	언제 배우나
DDL	틀(테이블)을 만들고 고치고 없앤다	`CREATE` · `ALTER` · `DROP`	본 것: CREATE TABLE · 2주차 도전
DML	행을 꺼내고 넣고 고치고 지운다	`SELECT` · `INSERT` · `UPDATE` · `DELETE`	**오늘 SELECT · 2주차 나머지**
DCL	권한을 주고 거둔다	`GRANT` · `REVOKE`	이름만 (SQLite에는 없음)
TCL	여러 문장을 한 묶음으로	`BEGIN` · `COMMIT` · `ROLLBACK`	2주차 트랜잭션

오늘은 SELECT 하나 — 꺼내 보기만 하니 DB가 바뀌지 않는다

SELECT의 기본 꼴: 열을 고르고 표를 정한다

예시 질의

```
SELECT name, nationality, birth_year
FROM author;
```

실행 결과

name	nationality	birth_year
한강	한국	1970
조남주	한국	1978
손원평	한국	1979
김호연	한국	1974
이미예	한국	1990
… 42행 더 (모두 47행)		

SELECT 열, 열, … — 보고 싶은 열 · 모든 열은 *

FROM 테이블 — 어느 표에서

키워드는 대소문자 무관 — select = SELECT

끝에 ; — 문장이 여럿일 때 나눔

결과: library.sql을 SQLite로 실행 (앞 5행만 옮김)

WHERE: 조건에 맞는 행만

예시 질의

```
SELECT name, birth_year
FROM author
WHERE nationality = '미국';
```

쓰는 말	뜻
`=` `<>` `!=`	같다 · 같지 않다
`<` `<=` `>` `>=`	작다 · 크다
`AND` `OR` `NOT`	그리고 · 또는 · 아니다
`BETWEEN a AND b`	a 이상 b 이하
`IN ('미국', '영국')`	여럿 가운데 하나

실행 결과

name	birth_year
재레드 다이아몬드	1937
마이클 샌델	1953
칼 세이건	1934
레이첼 카슨	1907
이민진	1968
어니스트 헤밍웨이	1899
제임스 클리어	*NULL*
리처드 하워드	1929

8행

글자는 '작은따옴표', 숫자는 그대로 · 결과: library.sql을 SQLite로 실행

ORDER BY: 정렬

예시 질의

```
SELECT name, birth_year
FROM author
WHERE birth_year < 1910
ORDER BY birth_year;
```

쓰는 법	뜻
`ORDER BY 열`	작은 것부터 (ASC, 기본)
`ORDER BY 열 DESC`	큰 것부터
`ORDER BY 열1 DESC, 열2`	열1이 같으면 열2로

실행 결과

name	birth_year
헤르만 헤세	1877
어니스트 헤밍웨이	1899
앙투안 드 생텍쥐페리	1900
조지 오웰	1903
레이첼 카슨	1907

5행

ORDER BY가 없으면 순서는 보장되지 않는다

결과: library.sql을 SQLite로 실행 · 글자는 가나다(코드) 순으로 정렬된다

LIKE: 글자 일부로 찾기

예시 질의

```
SELECT name, city
FROM publisher
WHERE name LIKE '문학%';
```

패턴	뜻	맞는 예
'문학%'	'문학'으로 시작	문학동네 · 문학수첩
'%문학%'	어디든 '문학'	(가운데·끝도)
'_학%'	둘째 글자가 '학'	문학동네

실행 결과

name	city
문학동네	파주
문학수첩	파주
문학사상	*NULL*

3행

% = 몇 글자든 · _ = 한 글자 — 목록 검색의 절단 검색

결과: library.sql을 SQLite로 실행

IS NULL: 비어 있는 값 찾기

✕ 이렇게 쓰면

```
SELECT name FROM publisher
WHERE city = NULL;
```

→ 0행 (오류는 아니지만 아무것도 안 나온다)

○ 이렇게 쓴다

```
SELECT name FROM publisher
WHERE city IS NULL;
```

IS NULL · IS NOT NULL — 빈 요소 점검에 그대로 쓴다

실행 결과

name
나무옆의자
팩토리나인
이성과힘
문학사상
와이즈베리
에코리브르
쓰다

7행

결과: library.sql을 SQLite로 실행 (= NULL은 0행, IS NULL은 7행)

COUNT와 DISTINCT: 세기와 중복 빼기

예시 질의 ①

```
SELECT COUNT(*), COUNT(birth_year), COUNT(name_original)
FROM author;
```

실행 결과

COUNT(*)	COUNT(birth_year)	COUNT(name_original)
47	29	18

COUNT(*) = 모든 행 · COUNT(열) = 그 열이 NULL 아닌 행

예시 질의 ②

```
SELECT DISTINCT nationality
FROM author;
```

실행 결과

nationality
한국
프랑스
독일
영국
이스라엘
미국
일본
중국
브라질

9행 · COUNT(DISTINCT nationality) = 9

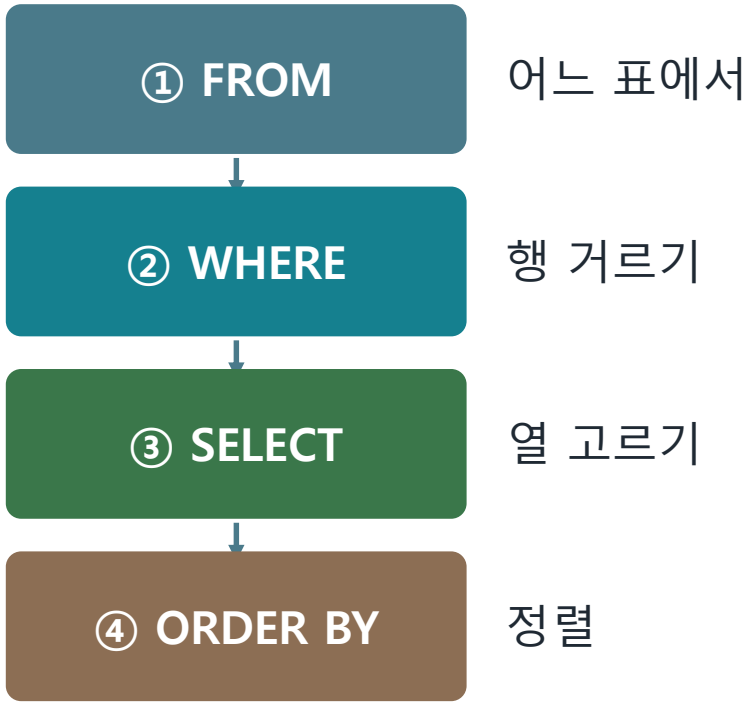
결과: library.sql을 SQLite로 실행 · ② 결과의 순서는 DB에 들어간 순서(ORDER BY 없음)

SELECT는 쓰는 순서와 도는 순서가 다르다

쓰는 순서 (38번 예시)

```
SELECT name, birth_year        -- ③
FROM author                    -- ①
WHERE birth_year < 1910        -- ②
ORDER BY birth_year;           -- ④
```

도는 순서



읽을 때는 FROM부터 · 쓸 때는 SELECT → FROM → WHERE → ORDER BY

자주 나는 오류와 메시지

이렇게 쓰면	SQLite 메시지	고칠 곳
`SELECT titel FROM book`	`no such column: titel`	열 이름 철자
`SELECT * FROM books`	`no such table: books`	테이블 이름 철자
`SELECT title FORM book`	`near "book": syntax error`	FORM → FROM
`SELECT title, FROM book`	`near "FROM": syntax error`	마지막 열 뒤 쉼표
`… WHERE title = 어린 왕자`	`near "왕자": syntax error`	글자는 '작은따옴표'
`… WHERE title = '어린 왕자`	`unrecognized token: "'어린 왕자"`	닫는 따옴표

메시지가 가리키는 곳의 바로 앞을 본다 · 실습실은 한국어 풀이를 함께 보여 준다

메시지: library.sql을 Python sqlite3(SQLite 3.50.4)로 실행해 받음 · SQLite 판에 따라 문구가 조금 다를 수 있다

실습 2 · 35분

실습 2: SELECT 기초 8문제 (1-1 ~ 1-8)

번호	제목	쓰는 것	순서 채점
1-1	도서 테이블 전체 보기	`SELECT *`	
1-2	필요한 열만 고르기	열 이름	
1-3	조건과 정렬	`WHERE · ORDER BY`	○
1-4	글자 일부로 찾기	`LIKE`	
1-5	비어 있는 값(NULL)	`IS NULL`	
1-6	개수 세기	`COUNT(*)`	
1-7	중복 없이 보기	`DISTINCT`	
1-8	반납 안 된 대출	`IS NULL · ORDER BY`	○

막히면

- 힌트 보기 (문제마다 1~3개)
- 36~43번 예시를 본떠 쓰기
- 글자 값은 '작은따옴표'
- 빨리 끝나면 짝 돕기

실습 자료 parkjuhyun-jnu.github.io/materials/metadata-2026-2/sql-db/

실습실 lab.html · 구조 보기 structure.html · 브라우저 안에서 SQLite(sql.js)가 돈다 — 설치·가입 없음

실습실 쓰는 법: 실행 · 채점 · 초기화

1 문제 고르기
왼쪽 번호 · 힌트 보기

→

2 SQL 쓰기
테이블·열 이름을 누르면 입력

→

3 실행
Ctrl + Enter · 결과 표

→

4 채점
결과만 비교 (열 이름은 안 봄)

채점이 알려 주는 것 (실습실 메시지)

맞았습니다. 13행 · 2열이 기대 결과와 같습니다.
열 개수가 다릅니다. … 문제에서 보라는 열만 고르세요.
행 개수가 다릅니다. … 조건(WHERE·HAVING)이나 JOIN을 확인하세요.
값은 모두 맞는데 순서가 다릅니다. ORDER BY를 확인하세요.

DB 초기화

실습 DB를 처음 상태로 되돌린다. 무엇을 바꿨든 겁내지 말 것.

실습실 lab.html · '…'는 생략 · 맞힌 기록과 쓰던 SQL은 내 브라우저에만 남는다

생각 1 · 5분 · 채점 없음

생각 1: 왜 표를 나눌까 (짝 토론)

book 테이블에 출판사 이름을 바로 적지 않고 publisher_id 번호만 둔 까닭은 무엇일까요? '창비'를 '창비(주)'로 고쳐야 할 때를 떠올려 보세요.

고칠 때
몇 행을 고쳐야 하나?

넣을 때
책이 없는 출판사는?

지울 때
마지막 책을 지우면?

실마리: 6부의 세 가지 이상 · 전거 통제

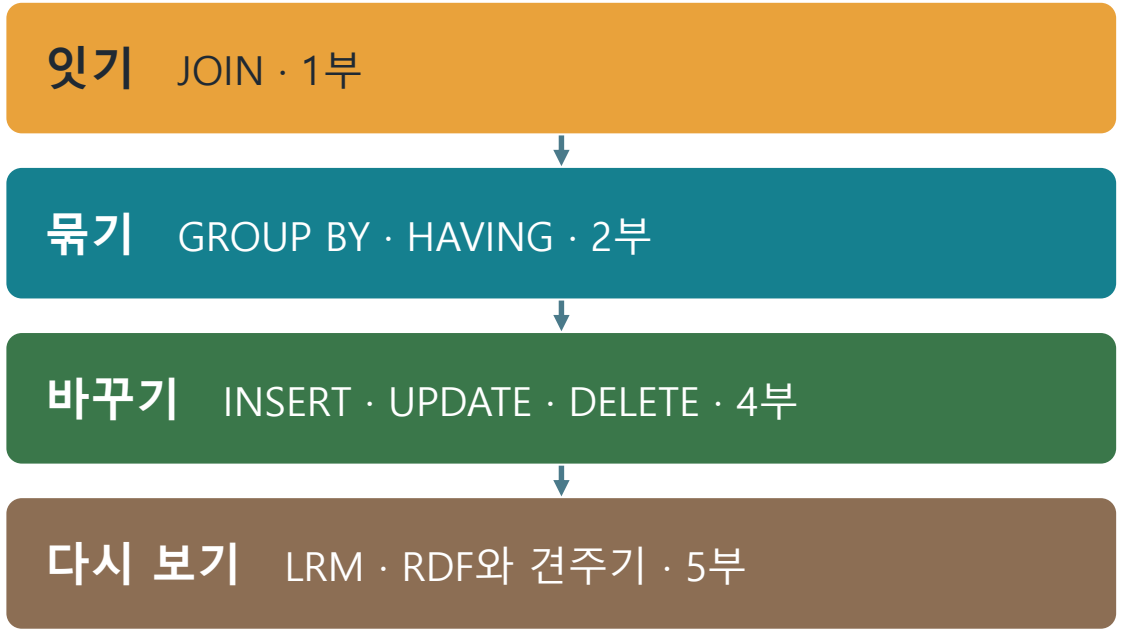
문제: 실습실 '생각 1' (exercises.json)

오늘 정리와 다음 주 예고

오늘 (3강 ①): 구조와 한 테이블 질의

- DB · DBMS · 데이터 모델
- 테이블 · 행 · 열 · 도메인 · NULL
- 기본 키 · 외래 키 · ERD (1:N · N:M)
- 정규화 (1NF~3NF) = 한 번만 적기
- SELECT · WHERE · ORDER BY · LIKE · IS NULL · COUNT

다음 주 (3강 ②): 잇기



다 못 푼 문제는 실습실에서 이어서 (기록은 내 브라우저에 남는다)

실습 자료: parkjuhyun-jnu.github.io/materials/metadata-2026-2/sql-db/

감사합니다

질문과 의견을 자유롭게 나눠 주세요

전남대학교 문헌정보학과 부교수 박주현 · park51566@jnu.ac.kr